THE REDEMPTION — CLOUD ENGINEER ASSESSMENT

Topic: "The Redemption" — a business-critical microservice on AWS EKS that handles global hotel loyalty-point deductions.
Scope: Answers assessment areas A–E plus the disaster-recovery design, and documents every Terraform component and why it was chosen.

0. SCENARIO & SERVICE-LEVEL OBJECTIVES

Traffic: Steady baseline with sudden 10x spikes (flash sales).
Constraint: Zero-downtime target; a failed deduction is direct revenue loss and a double-spend is a correctness incident.
Regions: PRIMARY = Thailand (Bangkok, ap-southeast-7).
STANDBY = Singapore (ap-southeast-1).

SLOs (drive every decision below):

- Availability (non-5xx) 99.95% / 30d -> 21m 36s error budget
- Latency p99 < 500 ms
- Idempotency correctness 100% (hard SLO — no double deduction)

Burn-rate alerting: 14.4x fast-burn pages immediately; 3x slow-burn opens a ticket. Rules live in modules/observability and the in-cluster Prometheus Rule.

1. ARCHITECTURE OVERVIEW

Edge → Route 53 (global DNS + health-based failover) → CloudFront + Shield → WAF v2 → regional ALB → EKS (Karpenter-scaled pods) → data tier (RDS PostgreSQL, DocumentDB, ElastiCache Redis, S3, MSK Kafka).

Why this shape:

- EKS + Karpenter, not a fixed ASG: Karpenter provisions right-sized nodes in around 30 to 60s versus 3–5 min for an ASG-backed Cluster Autoscaler. At a 10x flash spike that delta is the difference between graceful scale-out and a 503 storm. Karpenter also bin-packs and uses Spot with On-Demand fallback, cutting the bursty-tier compute bill around 60 to 70%.
- Active-standby across two regions with PER-SERVICE Global DNS: the app only ever talks to db.<zone>, mongo.<zone>, cache.<zone> — never a regional endpoint. On failover the DNS record flips and the app reconnects with no code or config change. This is cheaper and far less error-prone than active-active (which would need conflict resolution on point balances — the exact place a double-spend bug is unacceptable).
- Managed data services over self-hosted: RDS, DocumentDB, ElastiCache, MSK and S3 are all managed so the 3-person team spends its time on the product, not on operating stateful clusters across regions.

2. TERRAFORM COMPONENT INVENTORY (modules/ + per-region root files)

Every module is single-region and split by purpose. Each region instantiates a module via its own root file (eks-th.tf / eks-sg.tf, …). Replication that needs both regions (RDS replica, S3 CRR, DocumentDB global cluster) is wired at the root where both providers are in scope.

MODULES

modules/kms

One customer-managed key per data domain (eks, rds, docdb, redis, s3, secrets, ebs, msk, logs), rotation on, scoped key policy. Every other module is handed the ARN it needs — nothing uses an aws/* default key.

modules/vpc

3-AZ VPC: public / private-app / private-data subnet tiers, one NAT per AZ (an AZ loss can't sever egress for the survivors), S3+DynamoDB gateway endpoints and interface endpoints (ECR/STS/Secrets/KMS/Logs/Monitoring) so AWS-API traffic never touches a NAT, and REJECT flow logs. SPLIT ADDRESS SPACE: nodes get 10.x.x.x (private_app subnets); pods get 100.x.x.x from a 100.64.0.0/16 SECONDARY CIDR via VPC CNI custom networking + per-AZ ENIConfig. Keeps the 10.x.x.x space lean and node/pod IPs cleanly separable.

modules/security-group

Reusable per-component SG. Ingress is by SOURCE SECURITY GROUP (the EKS pod/cluster SG) — not CIDR — so the rule keeps working across the 100.x pod CIDR and means each data store only accepts connections FROM EKS PODS. rds, docdb, redis and kafka each instantiate one.

modules/eks

Cluster (KMS-encrypted secrets, control-plane audit logs), OIDC provider, system node group, Karpenter controller IRSA + spot-interruption SQS, managed ADD-ONS (vpc-cni & ebs-csi bound to dedicated least-privilege IRSA roles, kube-proxy, coredns, eks-pod-identity-agent),

EKS ACCESS ENTRIES (admins→ ClusterAdmin, developers → namespaced Edit), SSM node access for break-glass (no SSH/bastion), and POD IDENTITY associations.

modules/irsa

Reusable IAM-Role-for-ServiceAccount: trust scoped to one namespace/SA on one cluster's OIDC. Used for External Secrets and the AWS Load Balancer Controller.

modules/secrets

Secrets Manager secrets, KMS-encrypted, optional cross-region replica for DR. External Secrets pulls them into the cluster; the app's role is granted GetSecretValue on these ARNs only.

modules/rds

PostgreSQL with a dedicated PARAMETER GROUP (force_ssl, statement logging), Multi-AZ, RDS-managed master password in Secrets Manager, and a cross-region-replica mode (replicate_source_db_arn) for the SG copy.

modules/docdb

DocumentDB regional cluster that joins a global cluster (primary in TH, secondary in SG), KMS-encrypted.

modules/redis

ElastiCache Redis (cluster mode, TLS, at-rest KMS). Hot in TH region; an empty cold-standby copy in SG region.

modules/s3

Versioned, SSE-KMS, public-access-blocked bucket. CRR and the Multi-Region Access Point are wired in s3-th.tf.

modules/kafka

MSK (IAM SASL, TLS, KMS, open monitoring). Independent cluster per region.

modules/waf

Regional WAFv2 ACL: AWS managed Common + KnownBadInputs + a 2000-req/5-min/IP rate rule. Associated to the ALB by the LB Controller via Ingress annotation.
modules/ecr
Immutable, scan-on-push, KMS-encrypted repo with registry cross-region replication to SG.
modules/observability
Amazon Managed Prometheus (+ SLO recording/burn rules) and Grafana, plus app/audit log groups. Runs in SG region so it survives a TH region outage.

ROOT (per region)

versions, providers (aws=TH default, aws.sg alias), locals, variables (grouped per module, region-suffixed), then one file per module per region: kms-th/sg, vpc-th/sg, eks-th/sg, iam-th/sg, secrets-th, rds-th/sg, docdb-th/sg, redis-th/sg, s3-th/sg, kafka-th/sg, ecr, waf-th/sg, security-th/sg, observability, route53, outputs.

KMS + SECRETS INTEGRATION (cross-cutting)

- Every at-rest store (EKS secrets, RDS, DocumentDB, Redis, S3, MSK, EBS, CloudWatch Logs, ECR) encrypts with a customer-managed key from modules/kms.
- The DB master password is created and rotated by RDS directly into Secrets Manager. App runtime secrets live in modules/secrets, replicated to SG.
- The app reads secrets at runtime via External Secrets (IRSA) and via its Pod Identity role; both are scoped to redemption/* and the specific KMS keys.

A. COMPUTE & ARCHITECTURE

EKS 1.30 with a small tainted system node group (CoreDNS, kube-proxy, Karpenter, LB Controller) and Karpenter for all application capacity. Baseline 6 app replicas, two per AZ, with hard pod anti-affinity per host + topology spread, so a full-AZ loss never drops quorum. Rollouts use maxUnavailable = 0 + a presto drain matched to ALB deregistration so deploys are 502-free.

Resilience: pod crash → liveness restart; node/spot loss → Karpenter SQS interruption pre-drains and replaces in <2 min; AZ loss → ALB + topology spread shift traffic and Karpenter scales the survivors; bad deploy → Argo Rollouts canary auto-aborts on an error rate/latency analysis.

B. SCALABILITY (the 10x spike, automatically)

Three layers scale together:
App: HPA v2 on http_requests_per_second/pod (Prometheus Adapter) with a CPU safety net. RPS reacts in around 15s; CPU lags 30 to 60s — too slow for a flash sale. Aggressive scale-up (double or +20 pods / 15s, no stabilization), conservative 10-min scale-in (sales come in waves).
Nodes: Karpenter, Spot-first with On-Demand fallback, aroud 30 to 60s to Ready.

Data: RDS Proxy multiplexes connections so pod fan-out doesn't exhaust Postgres; read replicas for lookups; Redis cluster mode (3 shards); DocumentDB scales reads on the global cluster. Known events (scheduled sales) also pre-warm minReplicas 15 min ahead.

C. SECURITY & NETWORKING (least privilege + defense in depth)

Edge: CloudFront + Shield; WAF managed rules + per-IP rate limit.
Transport: TLS 1.3 at the ALB; in-cluster TLS to data stores.
Network: pods in private subnets only; default-deny NetworkPolicy; data subnets isolated; VPC endpoints keep AWS-API traffic off the NATs. Nodes 10.x.x.x / pods 100.x.x.x (secondary CIDR). Every data store has its own security group that allows ingress ONLY from the EKS pod security group (SG-reference, not CIDR) — least privilege at L3/L4.
Identity: EKS ACCESS ENTRIES (no aws-auth ConfigMap) — admins get ClusterAdmin, developers get namespaced Edit. POD IDENTITY gives the app its AWS role with no static keys; IRSA backs the controllers. SSM Session Manager is the only node-shell path (logged, no SSH, no bastion).
Data: all stores KMS-encrypted with customer-managed keys; Secrets Manager + External **Secrets for credentials**; RDS-managed rotating master password.
Cluster: private API endpoint by default; restricted Pod Security Standard; read-only rootfs, drop ALL caps, runAsNonRoot, IMDSv2 hop-limit 1.
Supply: ECR immutable tags + scan-on-push; admission policy rejects unsigned or vulnerable images (Kyverno, with a 2-approver break-glass).
Detect: GuardDuty + Security Hub (AWS Foundational) per region → PagerDuty; VPC flow logs (REJECT) + EKS audit logs to CloudWatch.

D. RELIABILITY & OBSERVABILITY

SLI/SLO with three signals: availability ratio, p99 latency, saturation. Metrics via Prometheus → Amazon Managed Prometheus (SG); dashboards in Managed Grafana; traces via OpenTelemetry → X-Ray (tail-sample 100% errors, 5% success); logs via Fluent Bit → CloudWatch; a CloudWatch Synthetics canary exercises a full redeem→refund cycle every 60s from three regions. Burn-rate alerts (not raw thresholds) drive paging. Recovery: AZ loss is automatic; bad deploy auto-rolls back; region loss is runbook RD-01 (below).

E. OPERATIONS

Day-2 toil reduction:

- GitOps (ArgoCD): merge = deploy; no manual kubectl.
- Karpenter expires after 720h recycles nodes onto fresh patched AMIs.
- RDS-managed + Secrets Manager rotation; ACM auto-renew; cert-manager.
- Every alert carries a runbook link; top-3 alerts auto-remediate via Lambda.
- Monthly chaos game-day (AWS FIS): kill an AZ, terminate pods, throttle NAT.

Team split (1 Senior + 2 Juniors):
Senior SRE — owns architecture & gatekeeping: VPC, EKS control plane, IAM/IRSA/Pod Identity, KMS, WAF, GuardDuty; SLOs & error-budget policy; DR runbook RD-01 & game-day; Terraform merge gate.
Junior A — workload layer: Kubernetes manifests, HPA tuning + k6 load tests for the 10x scenario, Argo Rollouts canary.
Junior B — observability & automation: Prometheus rules, Grafana dashboards, X-Ray, log routing, synthetics, auto-remediation Lambdas.

Both juniors join the on-call shadow rota in week 3; primary on-call rotates off the Senior after the first clean game-day.

DISASTER RECOVERY (active-standby, Bangkok → Singapore)

Targets: RTO <= 30 min, RPO <= 1 min. Quarterly game-day verified.

Service	Mechanism	DR Steady State	RPO
EKS / App	None (stateless)	Up, app scaled to 0	N/A
ECR Images	Registry cross-region replication	Mirrored	Seconds
RDS PostgreSQL	Cross-region asynchronous read replica	Receiving WAL	< 1 minute
DocumentDB	Global cluster (managed)	Secondary online	Seconds
Redis	None – cold standby (empty)	Up, empty	N/A*
S3	Cross-Region Replication (CRR) + Replication Time Control (RTC) 15-minute SLA + Multi-Region Access Point (MRAP)	Receiving copies	< 15 minutes
MSK Kafka	None – cold standby (empty)	Up, empty	Accepted*
Secrets	AWS Secrets Manager replication	Replicas ready	Minutes
Terraform State	S3 backend in Singapore + CRR	N/A	Seconds

* Recovery requires rebuilding or replaying data from upstream sources

Why Redis & Kafka are cold: their contents are derivable from the source-of-truth databases. Paying for cross-region replication of a cache (or running MirrorMaker for an event log whose truth lives in RDS) is wasted spend; we warm Redis from RDS and replay Kafka from the last offset on failover instead.

Runbook RD-01 (executable inside the 30-min budget):

1. Confirm regional impact (Service Health + 3 external R53 health-check regions agree) — avoids false failover on a local blip. (~3 min)

2. Promote stateful in parallel: rds promote-read-replica; docdb failover-global-cluster; S3 needs nothing . (~5 min)
3. Reshape SG EKS: patch Karpenter limits.cpu 100→2000; run the redis-preload Job to warm hot keys from RDS. (~5 min)
4. Swap the Kafka bootstrap ConfigMap (ArgoCD overlay). (~30 s)
5. Re-point ArgoCD at SG; app Deployment + HPA scale out. (~5 min)
6. Verify Route 53 flipped: every hostname now resolves to SG. (passive)
7. Smoke test via the synthetic canary; declare DR active.

Fail-back (RD-02) is done in a planned window, never under pressure.

DEPLOYMENT (two var-files per environment — one per region)

```
cd terraform
terraform init
terraform apply \
-var-file=environments/prod-th.tfvars \
-var-file=environments/prod-sg.tfvars
```

Each tfvars file is sectioned per module (vpc / eks / rds / docdb / redis / kafka / s3). The -th file also holds shared globals (project, dns, principals). Route 53 failover records are created on a second apply once the ALB DNS names are known (populate the `alb` variable). dev sets enable_dr=false.

KEY TRADE-OFFS (stated, not hidden)

- Active-standby over active-active — avoids cross-region write-conflict risk on point balances; accepts a 30-min RTO.
- Cold Redis & Kafka — saves around 3000$+ per month for regenerable data; accepts a around 5-min warm-up on the rare failover.
- Aggressive HPA scale-up may over-provision ~10–20% for the first 60s of a spike — cheaper than burning error budget by under-provisioning.
- Two-step apply for Route 53 (ALB DNS only exists after the Ingress) — in a real pipeline this is one ArgoCD post-sync hook + a second apply.